Working with data in Python 2

Python Programming for Business Intelligence

Sippo Rossi
April 1st 2026

Learning goals

Theoretical:

- Understand the ETL process
- Know what are common data sources
- Know what are common data formats outside of Python
- Know what APIs are

Practical:

- Know how to connect Jupyter Notebooks to various data sources
- Know how to use SQL within Python
- Know how to connect to an API
- Know how to join, merge and reshape data with Python

Motivation

Most of the time when you work with Jupyter Notebooks, your data will be coming from some external source(s) such as CSV files, databases and/or an API

Knowing how to connect to these data sources and combine data from multiple sources is one of the most important basic skills in business intelligence

Agenda

- The ETL process
- Loading data: flat files
- Connecting to databases
- Application programming interfaces (APIs)
- Joining data and reshaping data

The ETL process

What is ETL?

ETL stands for extract, transform, load

Extract relevant data from sources such as databases, the ERP or CRM

Transform the data into a desired format so that it is easy to work with

Load the data into the target system (typically a data warehouse)

Read more here: https://aws.amazon.com/what-is/etl/

The business intelligence process visualized

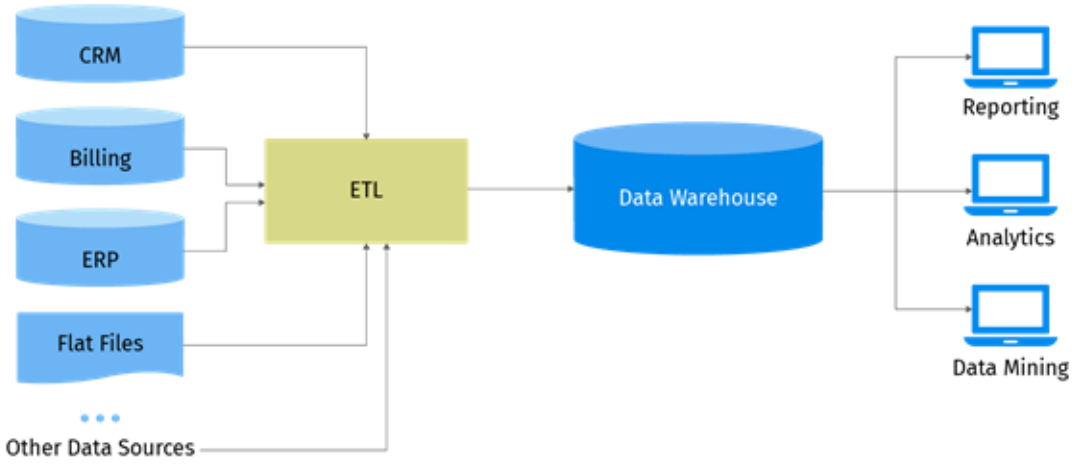


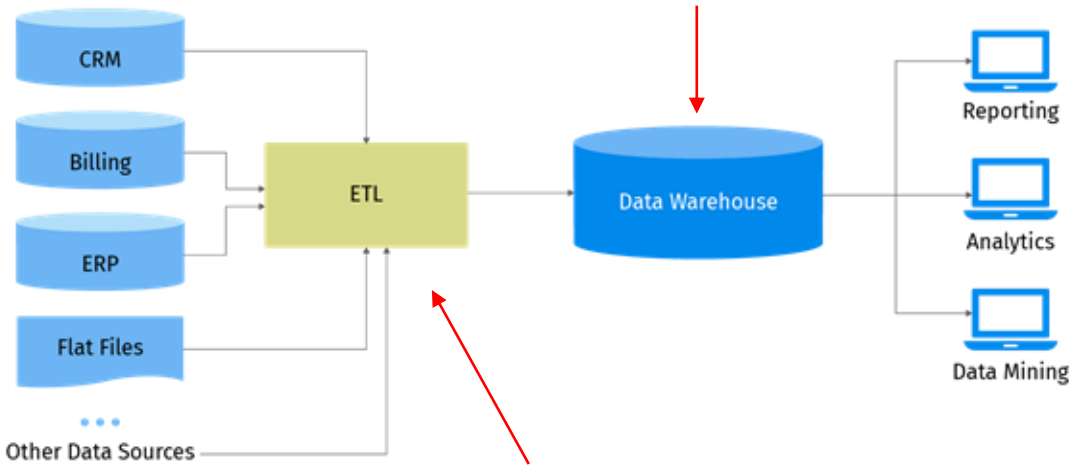
Image from: https://blog.griddynamics.com/edw-performance-comparison/

The business intelligence process visualized

Typically, the data we get from the ETL process is stored in a data warehouse. In this course we are limited to a Jupyter Notebook

Databases or external software where you can retrieve data from

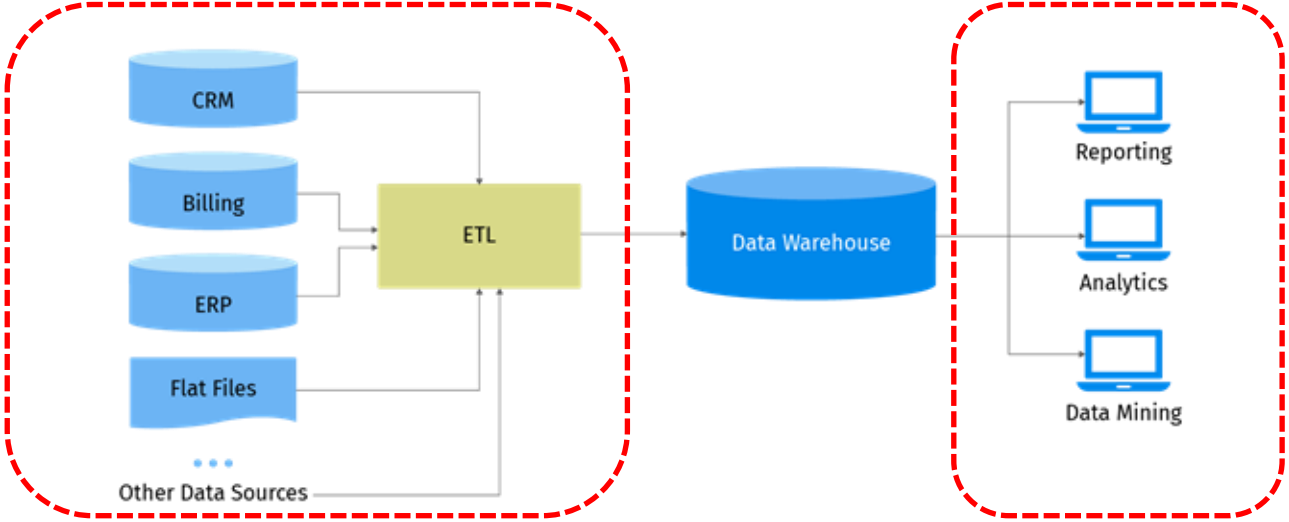
CSV files, JSON files, connecting to an API



This can be done with a mix of Python and SQL scripts (or with ETL software)

Image from: https://blog.griddynamics.com/edw-performance-comparison/

The business intelligence process visualized



Today we focus on this

Later we focus on these

Image from: https://blog.griddynamics.com/edw-performance-comparison/

ETL in this course

In business intelligence, unless you are only the end user of a dashboard or excel made by someone else, ETL is one of the most important steps

In this course, we do not set up a data warehouse, so for practical purposes we are doing ETL and storing the results in the same Jupyter Notebook where we do the ETL process

Loading data

Loading data with NumPy and Pandas

Often when you start working within Jupyter Notebooks, one of the first tasks is to load data

One of the most common formats for sharing tabular data are .csv (comma separated values) files

While excel is common too, csv is always better if you can choose between the two

Loading data with NumPy

Note that in a business context, data would most likely be loaded with Pandas rather than NumPy when the data has more than one data type present

To load csv files with NumPy use genfromtxt():

```
np.genfromtxt('path/file.csv', delimiter=',')
```

You can also specify how to deal with missing values with the missing_values parameter

Inspecting data with NumPy

After loading data into a new environmentm it is a good idea to check that everything was done correctly. For example:

Check dimensions:

```
ndarray.shape
```

Check the number of elements:

```
ndarray.size
```

Check the datatype (to change it use ndarray.astype):

```
ndarray.dtype
```

Loading data with Pandas

Pandas supports efficiently reading and writing data between in-memory data structures and different formats: CSV and text files, Microsoft Excel, SQL databases

The primary way to import flat files is with read_csv()

```
pd.read_csv('path/file.csv')
```

Inspecting data with pandas

Check the first or last 5 rows with df.head() or df.tail():

```
df.head()
df.tail()
```

Check the basic information of the datatypes, memory usage and etc with info:

```
df.info()
```

Check the shape of the Dataframe with the shape attribute:

```
df.shape
```

Check the names of the columns with the respectively named attribute:

```
df.columns
```

Inspecting data with pandas

It is also a good idea to check the number of missing values.

To get a count of how many null values per column there are:

```
df.isnull().sum()
```

Or if you want to get a count of all the different values in a column including missing values:

```
df[feature].value_counts()
```

Lastly, summary statistics can be viewed with describe:

```
df.describe()
```

Challenges with Encodings in Multilingual Datasets

Finnish/Swedish Characters:

- Letters like å, ä, ö are common in Finnish and Swedish
- If a CSV file is not saved in UTF-8 encoding, these characters may appear as garbled text (e.g., Ã¤, Ã¶).
- Always specify encoding='utf-8' when reading/writing files.

Common issues with CSV files

Varying practices for decimals

- Some countries use **comma (,)** as a decimal separator (e.g., 3,14) while others use **dot (.)** (e.g., 3.14).

Varying delimiter

- Many CSVs use **semicolons (;)** as delimiters, especially if commas are used for decimals. Other possible options include pipes | and tabs

Exporting CSV files

Naturally both NumPy and pandas provide straightforward functions for exporting your data as CSV files (and other formats)

```
df.to_csv('path/file.csv', index=False)
```

The file gets stored to the current working directory if no path is specified (if using just 'file.csv')

JSON files

JavaScript Object Notation (JSON) is a common data format especially for data retrieved from web APIs

JSON data consists of human-readable name-value pairs and arrays

Python's built-in json module handles raw JSON, while pandas can read JSON directly into a DataFrame

Loading data with NumPy and Pandas

Going through *L3_loading_data.ipynb*

Connecting to databases

Recap of databases

Relational databases are used to store structured data in tables that have rows and columns

Most relational databases can be queried with SQL (Structured Query Language) and we can embed SQL queries within Python code that is used to interact with databases

If you have not taken a database course, I suggest you use AI to help you write the SQL queries that you include in your Python code

In this course we will use SQLite databases

Why connect to a database with Python?

Often data is stored in databases rather than flat files such as csv files

Being able to bring data from a database to a Python environment rather than by storing it in an inconvenient intermediate format is much more efficient

Conversely being able to move data from a Python program or other interface back into a database is also valuable

Databases and Python

Databases are great for storing and accessing massive amounts of data and a lot of things can be accomplished with SQL

However, SQL has a limited number of built-in functions when compared to Python and its libraries

Furthermore, conducting data analysis is much easier with Python

Thus, while data should be stored in a database and initial preprocessing can be done with SQL, for a lot of tasks moving to Python is necessary

Connecting to a database

Python supports connecting to common commercial and open-source databases

Through these database interfaces, Python supports running SQL commands and accessing, modifying and querying from a database through a Python script

For a full list of database interfaces available for Python:
https://wiki.python.org/moin/DatabaseInterfaces

Connecting to a SQLite database

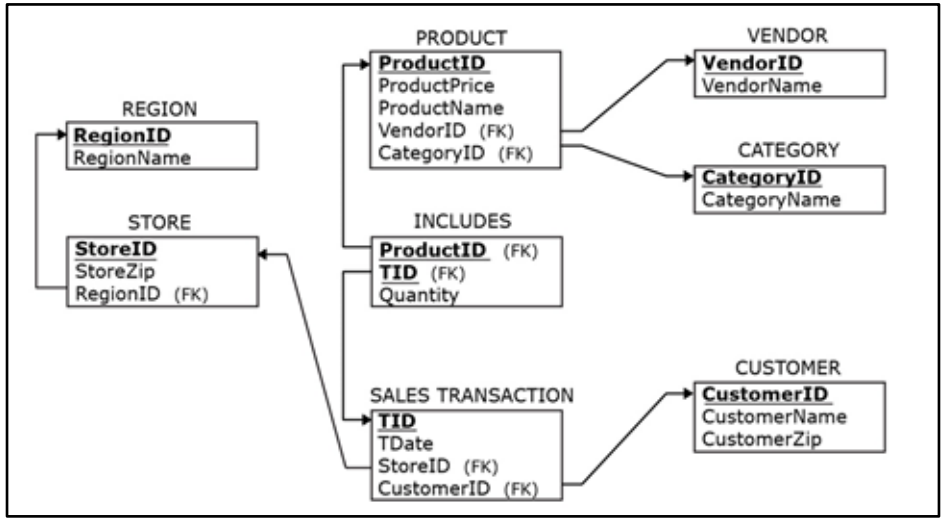
SQLite databases can be accessed with the sqlite3 module

Simply include the following in the beginning of the script:

```
import sqlite3
db = sqlite3.connect('path/retail.db')
```

Now the python file has access to the specified database. Today the examples will use the same retail company example as previously

Retail Company Sales Department Database



Interacting with a SQLite database

The method *execute* can be used to send SQL commands to the database.

Querying the database:

```
db.execute("SELECT MAX(ProductPrice) FROM Product").fetchone()
```

Adding a record to the database:

```
db.execute(”INSERT INTO Vendor (VendorName) VALUES ('Nike')")
```

Fetchone() vs fetchall()

In queries we specify using the method *fetchall* or *fetchone* how the output should be

fetchall() retrieves the results of the query as a list where each returned row is a tuple

fetchone() retrieves the first row of the result of the query as a tuple

Fetchone() vs fetchall() input and output

Input:

```
db.execute("SELECT MAX(ProductPrice) FROM Product").fetchone()
```

Result:

(500,)

Input:

```
db.execute("SELECT ProductName, ProductPrice) FROM Product LIMIT 3").fetchall()
```

Result:

[('Zzz Bag', 100), ('Easy Boot', 70), ('Cosy Sock', 15)]

Parameters

Parameters can be used to allow feeding values into the query:

```
name = input("Product name: ")
price = db.execute("SELECT ProductPrice FROM Product WHERE ProductName=?", [name]).fetchone()
```

This program will ask you the name of the product and then input that name to the WHERE clause of the query

Parameters

Multiple parameters can be give:

```
zip = input("Store zip number: ")
region = input("Store region ID: ")
db.execute("INSERT INTO Store (storezip, regionid) VALUES (?, ?)", [zip, region])
```

This program asks for the zip code and region ID and based on the input adds a record to the Store table

Error handling

Note that proper error handling is recommended when writing Python programs

E.g. if the query fails for some reason, it might be ideal for the script to still continue running rather than being stopped due to the error. One approach is using try & except

```
try:
        db.execute("CREATE TABLE Vendor (Vendorid INTEGER PRIMARY KEY, vendorname TEXT)")
except:
        print("Cannot execute the given command")
```

This command would fail since the table already exists

Performance considerations

Many tasks can be completed either with SQL or Python and even within these there can be multiple ways to achieve the same result

In general, databases are very well optimized for operations on large datasets with many rows and columns and are better suited for heavy duty tasks than Python and its libraries

It is also better to reduce the number of separate commands accessing a database when possible and e.g. use one query rather than multiple simpler ones

Note though that not all operations are slow and purely reducing the number of lines of code does not immediately improve performance

Minimizing work outside of the database

These two approaches to calculating the most expensive product both work:

SQL:

```
mostExpensive = db.execute("SELECT MAX(ProductPrice) FROM Product").fetchone()
```

SQL and Python:

```
prices = db.execute("SELECT ProductPrice FROM Product").fetchall()
mostExpensive = max(prices)
```

But the first one is better as the second one brings potentially millions of rows of unnecessary data outside of the database. With this dataset, the execution time for the 2nd can be even 2.5x slower

Which one is better and why?

Approach 1:

```
data = db.execute("SELECT v.vendorid, COUNT(*) FROM vendor v LEFT JOIN product p
                   ON v.vendorid = p.vendorid GROUP BY v.vendorid").fetchall()

for row in data:
    print(row[0], row[1])
```

Approach 2:

```
vendors = db.execute("SELECT vendorid FROM Vendor").fetchall()

for vendor in vendors:
    amount = db.execute("SELECT COUNT(*) FROM Product WHERE vendorid=?", [vendor[0]]).fetchone()
    print(vendor[0], amount[0])
```

Minimizing number of queries

Avoid unnecessary queries. Below is an example of an inefficient approach for printing the number of products from each vendor:

```
vendors = db.execute("SELECT vendorid FROM Vendor").fetchall()

for vendor in vendors:
    amount = db.execute("SELECT COUNT(*) FROM Product WHERE vendorid=?", [vendor[0]]).fetchone()
    print(vendor[0], amount[0])
```

Here a total of 5 queries were made

Minimizing number of queries

This achieves the same result as the previous slide, but with only one query:

```
data = db.execute("SELECT v.vendorid, COUNT(*) FROM vendor v LEFT JOIN product p
                   ON v.vendorid = p.vendorid GROUP BY v.vendorid").fetchall()

for row in data:
    print(row[0], row[1])
```

With the given dataset, this can be even 6 times faster than the previous approach

Connecting to a database with Pandas

The previous example were based on using the sqlite3 library.

As an alternative, with Pandas, you can also read data from a database directly into a Dataframe. First we need to connect to the database as previously:

```
db = sqlite3.connect("path/retailcompanysales.db")
```

Querying the database is then done with read_sql_query:

```
df = pd.read_sql_query("SELECT * from Product", db)
```

Pandas vs sqlite3 in Python

When querying a database using the pd.read_sql_query, we can instantly get a DataFrame which is easier to work with than the list returned by sqlite3's execute

However, the list can also easily be turned into a DataFrame and sqlite3 module is faster than Pandas

Which one to use? It depends…

What affects performance

The size of the dataset

The RAM available on the device

Computational complexity of the operation or query. Avoid loops where possible

Indexing of a database

Connecting to a database

Going through *L3_databases.ipynb*

Application Programming Interfaces

What are application programming interfaces

Many organizations offer application programming interfaces (APIs) as a convenient and standardized way to access the data from their services

An API is a mechanisms that make it possible for several programs to communicate with each other

In very simple terms, an API allows you to write a Python program that for example retrieves some content from an online service

E.g., X's API: https://developer.x.com/en/docs/x-api can be used for interacting with X users and fetching content from X programmatically

Connecting to an API

Some APIs require using Python's Requests library, while others have their own dedicated libraries

Here are tutorials with examples:

- https://www.geeksforgeeks.org/python/get-financial-data-from-yahoo-finance-with-python/
- https://www.geeksforgeeks.org/python/python-api-tutorial-getting-started-with-apis/

API keys

Often APIs limit how much each user can interact with it

While some APIs allows anyone to connect to them without a profile, many require also using an API-key, which is a unique identifier that links your requests to an account (and often to you credit card if you pay for using the API)

In general, API keys are to be treated like a secret password. Never share your API-keys and never ever upload them to GitHub

Connecting to an API

Going through *L3_api.ipynb*

Joining and reshaping data

Joins

The standard join function in Pandas for doing joins similar to those in SQL is merge(), which can be used like below:

```
pd.merge(df1, df2, how='inner', left_on="id", right_on = "id")
```

Joins can also be done on row indexes rather than specific id columns.

Read more here about the function and a tutorial comparing merge and SQL here

Types of joins

ID	X1
1	a1
2	a2

ID	X2
2	b1
3	b2

Inner Join

ID	X1	X2
2	a2	b1

Outer Join

ID	X1	X2
1	a1	NA
2	a2	b1
3	NA	b2

Left Join

ID	X1	X2
1	a1	NA
2	a2	b1

Right Join

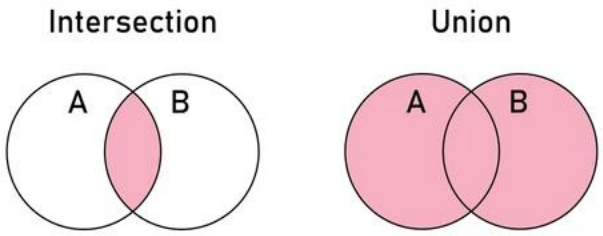
ID	X1	X2
2	a2	b1
3	NA	b2

Image from https://statisticsglobe.com/types-joins-for-pandas-dataframes-python

Union and intersection

Two common set operations are unions and intersections:

- pandas' concat() function can be used to combine datasets A and B that have the same columns, adding the rows together (union)
- pandas' merge can be used to return rows that are found in both datasets A and B (intersection)



Wide and long format (recap)

Data can be stored in a wide or long format

In the wide format each column represents a different variable

In the long format one column contains all the variables and another column gives the value for that variable

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Image from https://www.statology.org/long-vs-wide-data/

Wide and long format

Pandas has several methods that can be used to move from wide format to long format

melt() can unpivot a DataFrame from wide to long format

wide_to_long() is a method for specifically doing this

pivot() can be used to go from long to wide

Joining and reshaping data

Going through *L3_joining_reshaping.ipynb*

Conclusion

Conclusion

Today we touched upon the ETL process and how you can bring data from various sources into Jupyter Notebooks and how to modify the data into a shape that is easy to work with

If you are loading data to a Python environment, in most cases it will be done with Pandas especially when working with flat files

Whether the transforming of data makes sense on doing with Python or e.g. SQL, depends on the situation

It is a good idea to inspect the data always after loading before starting to work with it

Questions, comments suggestions?